

Business Case Study : Solution (Target SQL)

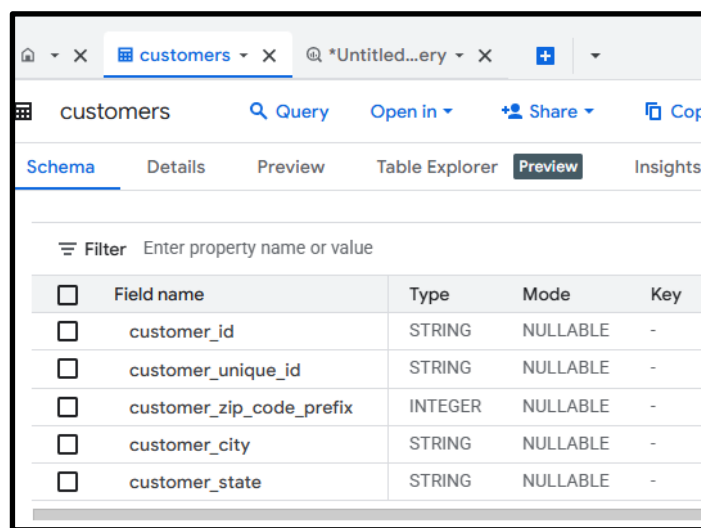
Submitted By: Heena Khan

- I. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:
 - A. Data type of all columns in the “customers” table.

DESCRIBE Target.customers;

OR

DESC Target.customers;

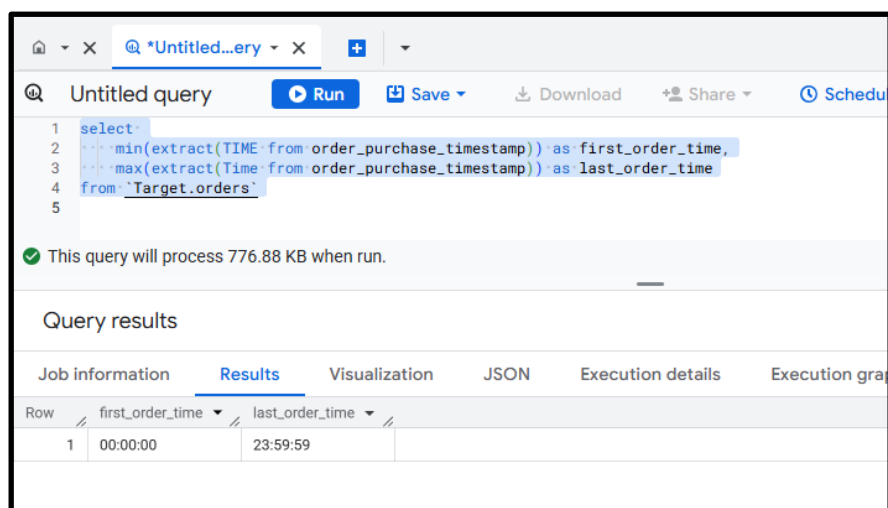


The screenshot shows a database interface with a tab for 'customers'. Below the tab, there are tabs for 'Schema', 'Details', 'Preview', 'Table Explorer', 'Preview', and 'Insights'. The 'Schema' tab is selected, showing a table with 5 columns: 'customer_id', 'customer_unique_id', 'customer_zip_code_prefix', 'customer_city', and 'customer_state'. Each column has a checkbox to its left. The 'Type' column shows 'STRING' for the first three and 'INTEGER' for the last two. The 'Mode' column shows 'NULLABLE' for all. The 'Key' column shows '-' for all.

Field name	Type	Mode	Key
customer_id	STRING	NULLABLE	-
customer_unique_id	STRING	NULLABLE	-
customer_zip_code_prefix	INTEGER	NULLABLE	-
customer_city	STRING	NULLABLE	-
customer_state	STRING	NULLABLE	-

- B. Get the time range between which the orders were placed.

```
select
    min(extract(TIME from order_purchase_timestamp)) as first_order_time,
    max(extract(Time from order_purchase_timestamp)) as last_order_time
from `Target.orders`
```



The screenshot shows a database interface with a tab for 'Untitled query'. Below the tab, there are buttons for 'Run', 'Save', 'Download', 'Share', and 'Schedule'. The 'Run' button is highlighted. Below the buttons, there is a text box containing the SQL query. Below the text box, there is a status bar indicating 'This query will process 776.88 KB when run.' Below the status bar, there is a section for 'Query results' with tabs for 'Job information', 'Results', 'Visualization', 'JSON', 'Execution details', and 'Execution gra'. The 'Results' tab is selected, showing a table with 2 columns: 'first_order_time' and 'last_order_time'. The table has 1 row with values '00:00:00' and '23:59:59'.

```
1 select
2     min(extract(TIME from order_purchase_timestamp)) as first_order_time,
3     max(extract(Time from order_purchase_timestamp)) as last_order_time
4 from `Target.orders`
5
```

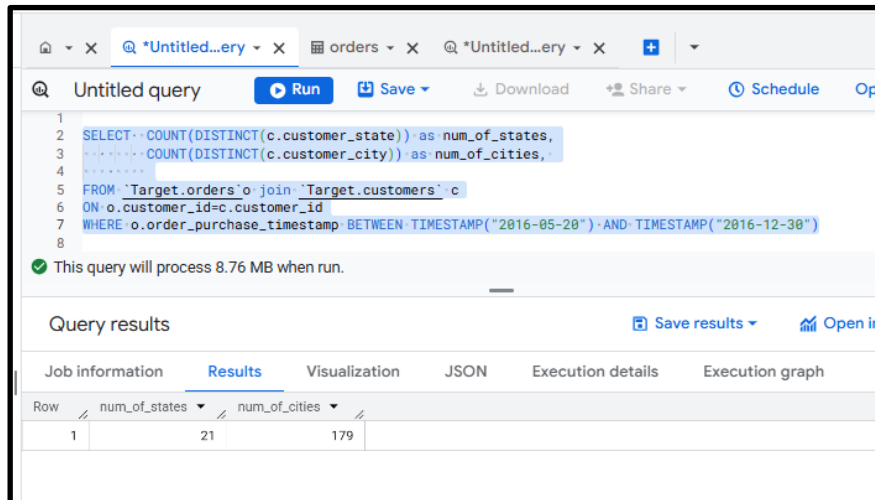
✓ This query will process 776.88 KB when run.

Query results

Job information	Results	Visualization	JSON	Execution details	Execution gra
Row	first_order_time	last_order_time			
1	00:00:00	23:59:59			

C. Count the Cities & States of customers who ordered during the given period.

```
SELECT COUNT(DISTINCT(c.customer_state)) as num_of_states,  
       COUNT(DISTINCT(c.customer_city)) as num_of_cities,  
  
FROM `Target.orders` o join `Target.customers` c  
ON o.customer_id=c.customer_id  
WHERE o.order_purchase_timestamp BETWEEN  
TIMESTAMP("2016-05-20") AND TIMESTAMP("2016-12-30")
```



The screenshot shows a SQL query editor with the following query:

```
1 SELECT COUNT(DISTINCT(c.customer_state)) as num_of_states,  
2       COUNT(DISTINCT(c.customer_city)) as num_of_cities,  
3  
4 FROM `Target.orders` o join `Target.customers` c  
5 ON o.customer_id=c.customer_id  
6 WHERE o.order_purchase_timestamp BETWEEN  
7       TIMESTAMP("2016-05-20") AND TIMESTAMP("2016-12-30")  
8
```

Below the query, it states: "This query will process 8.76 MB when run."

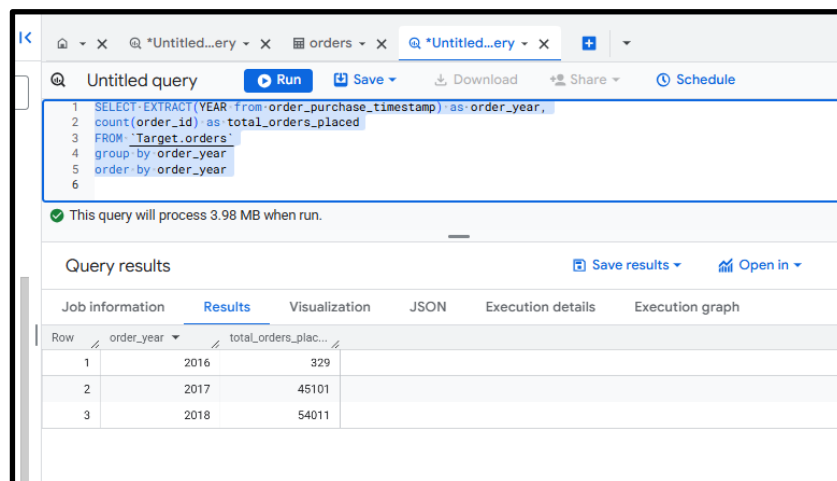
The query results are displayed in a table with the following columns: num_of_states and num_of_cities.

Row	num_of_states	num_of_cities
1	21	179

II. In-depth Exploration:

A. Is there a growing trend in the no. of orders placed over the past years?

```
SELECT EXTRACT(YEAR from order_purchase_timestamp) as order_year,  
       count(order_id) as total_orders_placed  
FROM `Target.orders`  
group by order_year  
order by order_year
```



The screenshot shows a SQL query editor with the following query:

```
1 SELECT EXTRACT(YEAR from order_purchase_timestamp) as order_year,  
2       count(order_id) as total_orders_placed  
3 FROM `Target.orders`  
4 group by order_year  
5 order by order_year  
6
```

Below the query, it states: "This query will process 3.98 MB when run."

The query results are displayed in a table with the following columns: order_year and total_orders_placed.

Row	order_year	total_orders_placed
1	2016	329
2	2017	45101
3	2018	54011

Trend Analysis:

1. Number of orders placed are drastically increased in year 2017 as compared to year 2016.
2. Number of orders placed are increased in year 2018 as compared to year 2017.

B. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```
SELECT EXTRACT(YEAR from order_purchase_timestamp) as order_year,
       EXTRACT(MONTH from order_purchase_timestamp) as order_month,
       count(order_id) as total_orders_placed
FROM `Target.orders`
group by order_year, order_month
order by order_year, order_month
LIMIT 10
```

Job information	Results	Visualization	JSON	Execution details	Execution g
Row	order_year	order_month	total_orders_plac...		
1	2016	9	4		
2	2016	10	324		
3	2016	12	1		
4	2017	1	800		
5	2017	2	1780		
6	2017	3	2682		
7	2017	4	2404		
8	2017	5	3700		
9	2017	6	3245		
10	2017	7	4026		

C. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

```
SELECT
CASE
  WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
  BETWEEN 0 AND 6 THEN "Dawn"
  WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
  BETWEEN 7 AND 12 THEN "Mornings"
  WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
  BETWEEN 13 AND 18 THEN "Afternoon"
  WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
  BETWEEN 19 AND 23 THEN "Night"
END as time_of_day,
count(*) as total_orders
FROM `Target.orders`
group by time_of_day
order by total_orders
```

Untitled query

Run Save Download Share

2 -- 0-6 hrs : Dawn
3 -- 7-12 hrs : Mornings
4 -- 13-18 hrs : Afternoon

✓ This query will process 776.88 KB when run.

Query results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	time_of_day	total_orders			
1	Dawn	5242			
2	Mornings	27733			
3	Night	28331			
4	Afternoon	38135			

III. Evolution of E-commerce orders in the Brazil region:

A. Get the month on month no. of orders placed in each state.

```
SELECT
    c.customer_state AS state,
    EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,
    EXTRACT(MONTH FROM o.order_purchase_timestamp) AS
order_month,
    COUNT(o.order_id) AS total_orders
FROM `Target.orders` o JOIN `Target.customers` c
ON o.customer_id = c.customer_id
GROUP BY
    c.customer_state,
    EXTRACT(YEAR FROM o.order_purchase_timestamp),
    EXTRACT(MONTH FROM o.order_purchase_timestamp)
ORDER BY
    state,
    order_year,
    order_month;
```

Query completed

Query results

Save results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	state	order_year	order_month	total_orders	
1	AC	2017	1	2	
2	AC	2017	2	3	
3	AC	2017	3	2	
4	AC	2017	4	5	
5	AC	2017	5	8	
6	AC	2017	6	4	
7	AC	2017	7	5	

Results per page: 50 1 - 50

Query completed

Query results

Save results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	state	order_year	order_month	total_orders	
23	AL	2017	2	12	
24	AL	2017	3	10	
25	AL	2017	4	23	
26	AL	2017	5	27	
27	AL	2017	6	10	
28	AL	2017	7	17	
29	AL	2017	8	18	
...	...	...	...	...	...

Results per page: 50 1 - 50

Trend Analysis:

To find out running growth trend (month-on-month % change in orders per state), we have to calculate the percentage change. Following is the query executed to get the percentage change.

```

WITH monthly_orders AS (
    SELECT
        c.customer_state AS state,
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,
        EXTRACT(MONTH FROM o.order_purchase_timestamp) AS order_month,
        COUNT(o.order_id) AS total_orders
    FROM `Target.orders` o JOIN `Target.customers` c
    ON o.customer_id = c.customer_id
    GROUP BY
        c.customer_state,
        EXTRACT(YEAR FROM o.order_purchase_timestamp),
        EXTRACT(MONTH FROM o.order_purchase_timestamp)
),
with_lag AS (SELECT
    state,
    order_year,
    order_month,
    total_orders,
    LAG(total_orders) OVER (PARTITION BY state ORDER BY order_year,
    order_month) AS prev_month_orders
    FROM monthly_orders)
SELECT
    state,
    order_year,
    order_month,
    total_orders,
    prev_month_orders,
    ROUND(SAFE_DIVIDE((total_orders - prev_month_orders) * 100.0, prev_month_orders), 2)
    AS mom_growth_percent
FROM with_lag
ORDER BY state, order_year, order_month;

```

Query results							
Job information Results Visualization JSON Execution details Execution gra							
Row	state	order_year	order_month	total_orders	prev_month_orders	mom_growth_perc...	
1	AC	2017	1	2	null	null	
2	AC	2017	2	3	2	50.0	
3	AC	2017	3	2	3	-33.33	
4	AC	2017	4	5	2	150.0	
5	AC	2017	5	8	5	60.0	
6	AC	2017	6	4	8	-50.0	

Query completed							
Query results							
Job information Results Visualization JSON Execution details Execution							
Row	state	order_year	order_month	total_orders	prev_month_orders	mom_growth_perc...	
20	AC	2018	8	3	4	-25.0	
21	AL	2016	10	2	null	null	
22	AL	2017	1	2	2	0.0	
23	AL	2017	2	12	2	500.0	
24	AL	2017	3	10	12	-16.67	
25	AL	2017	4	23	10	130.0	
26	AT	2017	4	22	22	0.0	

B. How are the customers distributed across all the states?

```

SELECT customer_state,
       count(*) as total_customer
FROM `Target.customers`
group by customer_state
order by total_customer
LIMIT 10;

```

Query results				
Job information		Results	Visualization	JSON
Row	customer_state	total_customer		
1	RR	46		
2	AP	68		
3	AC	81		
4	AM	148		
5	RO	253		
6	TO	280		
7	SE	350		
8	AL	413		
9	RN	485		
10	IL	485		

IV. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

A. Get the % increase in the cost of orders from year 2017 to 2018 *(include months between Jan to Aug only).*

You can use the “payment_value” column in the payments table to get the cost of orders.

SELECT

```
round(SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) =
2017 THEN payment_value END),2) AS total_2017,
round(SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) =
2018 THEN payment_value END),2) AS total_2018,
```

```
round((((SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) =
2018 THEN payment_value END) -
SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2017
THEN payment_value END))
/ SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2017
THEN payment_value END)) ,2)* 100 AS percent_increase
```

```
FROM `Target.payments` p join `Target.orders` o
ON p.order_id=o.order_id
```

```
WHERE EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 8
AND EXTRACT(YEAR FROM order_purchase_timestamp) IN (2017, 2018);
```

✓ Query completed				
Query results				Save results
Job information		Results	Visualization	JSON
Row	total_2017	total_2018	percent_increase	
1	3669022.12	8694733.84	137.0	

Analysis:

The percentage increase is 137% from year 2017 to 2018 in the first 8 months i.e. January to August.

B. Calculate the Total & Average value of order price for each state.

```
SELECT c.customer_state,
       count(o.order_id) as total_orders,
       round(avg(p.payment_value),2) as avg_order_price
FROM `Target.customers` c LEFT JOIN `Target.orders` o
ON c.customer_id=o.customer_id
LEFT JOIN `Target.payments` p ON o.order_id=p.order_id
group by c.customer_state
order by c.customer_state
LIMIT 10;
```

✓ This query will process 14.21 MB when run.

Query results				
Job information		Results	Visualization	JSON
Execution details				
Row	customer_state	total_orders	avg_order_price	
1	AC	84	234.29	
2	AL	427	227.08	
3	AM	154	181.6	
4	AP	70	232.33	
5	BA	3610	170.82	
6	CE	1398	199.9	
7	DF	2204	161.13	
8	ES	2107	154.71	
9	GO	2112	165.76	
10	MA	767	198.86	

Analysis:

1. In high volume states like BA,DF,ES,GO,PR,RJ,SC,SP (details taken from total_orders column), **need to increase sell (can be through marketing) as their average order price is low comparing to volume.**
2. States like AC, AM,AP,RR,AC,TO have low volume of orders but the average order price is good enough. **There might be Premium customers.**

C. Calculate the Total & Average value of order freight for each state.

```
SELECT
  c.customer_state AS state,
  round(SUM(ot.freight_value),2) AS total_freight_value,
  round(AVG(ot.freight_value),2) AS avg_freight_value
FROM `Target.customers` c LEFT JOIN `Target.orders` o
ON c.customer_id=o.customer_id LEFT JOIN `Target.order_items`
ot
ON o.order_id=ot.order_id
GROUP BY c.customer_state
ORDER BY c.customer_state
LIMIT 10;
```

1 --C. Calculate the Total & Average value of order freight for each state.				
Query completed				
Query results				Save results
Job information	Results	Visualization	JSON	Execution details
Execution graph				
Row	state	total_freight_value	avg_freight_value	
1	AC	3686.75	40.07	
2	AL	15914.59	35.84	
3	AM	5478.89	33.21	
4	AP	2788.5	34.01	
5	BA	100156.68	26.36	
6	CE	48351.59	32.71	
7	DF	50625.5	21.04	
8	ES	49764.6	22.06	
9	GO	53114.98	22.77	
10	MA	21522.77	22.26	

Analysis:

AM,AP,RN,RR have high average freight value considering the total freight value. (May be due to high shipping cost or remote orders).

V. Analysis based on sales, freight and delivery time.

- Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp.
- diff_estimated_delivery** = order_estimated_delivery_date - order_delivered_customer_date

```
SELECT
    order_id,
    DATE_DIFF(order_delivered_customer_date,
order_purchase_timestamp, DAY) AS time_to_deliver,
    DATE_DIFF(order_estimated_delivery_date,
order_delivered_customer_date, DAY) AS
diff_estimated_delivery
FROM `Target.orders`
WHERE order_delivered_customer_date IS NOT NULL
    AND order_estimated_delivery_date IS NOT NULL
    AND order_purchase_timestamp IS NOT NULL
LIMIT 10;
```

✓ This query will process 5.48 MB when run.

Query results

Job information	Results	Visualization	JSON	Execution details	Execution g
Row	order_id	time_to_deliver	diff_estimated_d...		
1	65d1e226dfaeb8cdc42f665422...	35	16		
2	770d331c84e5b214bd9dc70a1...	7	45		
3	dabf2b0e35b423f94618bf965fc...	7	44		
4	8beb59392e21af5eb9547ae1a9...	10	41		
5	2c45c33d2f9cb8ff8b1c86cc28...	30	28		
6	1950d777989f6a877539f53795...	30	-12		
7	bfb0f9bdef84302105ad712db...	54	-36		
8	3b697a20d9e427646d9256791...	23	0		
9	be5bc2f0da14d8071e2d45451...	24	10		

Results per page: 50

Analysis:

1. Many orders have positive diff_estimated_delivery, that means the actual delivery is delayed for that many days. Which can affect the business as customers can cancel orders, give poor ratings or return products.
2. Need to calibrate the estimated delivery or improve logistics.

- b. Find out the top 5 states with the highest & lowest average freight value.

```

SELECT
  CASE
    WHEN row_num_desc <= 5 THEN 'Highest'
    WHEN row_num_asc <= 5 THEN 'Lowest'
  END AS freight_type,
  state,
  avg_freight
FROM (
  SELECT
    c.customer_state AS state,
    round(AVG(oi.freight_value),2) AS avg_freight,
    ROW_NUMBER() OVER (ORDER BY AVG(oi.freight_value) DESC) AS
    row_num_desc,
    ROW_NUMBER() OVER (ORDER BY AVG(oi.freight_value) ASC) AS row_num_asc
  FROM `Target.order_items` oi JOIN `Target.orders` o
    ON oi.order_id = o.order_id
  JOIN `Target.customers` c
    ON o.customer_id = c.customer_id
  GROUP BY c.customer_state
)
WHERE row_num_desc <= 5 OR row_num_asc <= 5
ORDER BY freight_type DESC, avg_freight ASC;

```

Query results Save results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	freight_type	state	avg_freight		
1	Lowest	SP	15.15		
2	Lowest	PR	20.53		
3	Lowest	MG	20.63		
4	Lowest	RJ	20.96		
5	Lowest	DF	21.04		
6	Highest	PI	39.15		
7	Highest	AC	40.07		
8	Highest	RO	41.07		
9	Highest	PB	42.72		
10	Highest	RR	42.98		

Results per page: 50 1 – 10 of 10

- c. Find out the top 5 states with the highest & lowest average delivery time.

```

SELECT
  CASE
    WHEN row_num_desc <= 5 THEN 'Highest'
    WHEN row_num_asc <= 5 THEN 'Lowest'
  END AS type,
  state,
  avg_delivery_days
FROM (
  SELECT
    c.customer_state AS state,
    round(AVG(DATE_DIFF(o.order_delivered_customer_date,
    o.order_purchase_timestamp, DAY)),2) AS
    avg_delivery_days,
    ROW_NUMBER() OVER (ORDER BY
    AVG(DATE_DIFF(o.order_delivered_customer_date,
    o.order_purchase_timestamp, DAY)) DESC) AS
    row_num_desc,
    ROW_NUMBER() OVER (ORDER BY
    AVG(DATE_DIFF(o.order_delivered_customer_date,
    o.order_purchase_timestamp, DAY)) ASC) AS
    row_num_asc
  FROM `Target.orders` o
  JOIN `Target.customers` c
    ON o.customer_id = c.customer_id
  WHERE o.order_delivered_customer_date IS NOT NULL
  GROUP BY c.customer_state
) AS ranked_states
WHERE row_num_desc <= 5 OR row_num_asc <= 5
ORDER BY type DESC, avg_delivery_days ASC;

```

Query results				
Job information		Results	Visualization	JSON
Row	type	state	avg_delivery_days	
1	Lowest	SP	8.3	
2	Lowest	PR	11.53	
3	Lowest	MG	11.54	
4	Lowest	DF	12.51	
5	Lowest	SC	14.48	
6	Highest	PA	23.32	
7	Highest	AL	24.04	
8	Highest	AM	25.99	
9	Highest	AP	26.73	
10	Highest	RR	28.98	

- d. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

SELECT

```

c.customer_state as state,
ROUND(AVG(DATE_DIFF(o.order_delivered_customer_date,
o.order_estimated_delivery_date, DAY)), 2) AS
avg_diff_days
FROM `Target.orders` o JOIN `Target.customers` c
ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
AND o.order_estimated_delivery_date IS NOT NULL
GROUP BY state
ORDER BY avg_diff_days ASC
LIMIT 5;

```

Query results				
Job information		Results	Visualization	Execution
Row	state	avg_diff_days		
1	AC	-19.76		
2	RO	-19.13		
3	AP	-18.73		
4	AM	-18.61		
5	RR	-16.41		

Analysis:

1. The delivery speed of these 5 top states is **very good** (according to the estimated dates).
2. **Optimizing delivery estimates** in these states can increase accuracy.
3. These logistics practices can be followed **to improve performance in slower regions**.

VI. Analysis based on the payments:

- a. Find the month on month no. of orders placed using different payment types.

```
SELECT
    FORMAT_TIMESTAMP('%Y-%m', o.order_purchase_timestamp) AS year_month,
    p.payment_type,
    COUNT(DISTINCT o.order_id) AS total_orders,
FROM `Target.orders` o JOIN `Target.payments` p
    ON o.order_id = p.order_id
GROUP BY year_month, p.payment_type
ORDER BY year_month, p.payment_type;
```

Query results				
Job information		Results	Visualization	JSON
Execution details				
Row	year_month	payment_type	total_orders	
1	2016-09	credit_card	3	
2	2016-10	UPI	63	
3	2016-10	credit_card	253	
4	2016-10	debit_card	2	
5	2016-10	voucher	11	
6	2016-12	credit_card	1	
7	2017-01	UPI	197	
8	2017-01	credit_card	582	
9	2017-01	debit_card	9	
10	2017-01	voucher	33	
11	2017-02	UPI	200	

Analysis:

- Most popular payment method is "Credit/Debit Card".

- b. Find the no. of orders placed on the basis of the payment installments that have been paid.

```
SELECT
    payment_installments,
    COUNT(DISTINCT order_id) AS total_orders
FROM `Target.payments`
GROUP BY payment_installments
ORDER BY total_orders desc;
```

Query results			
Job information		Results	Visualization
Row	payment_installm...	total_orders	
1	1	49060	
2	2	12389	
3	3	10443	
4	4	7088	
5	10	5315	
6	5	5234	
7	8	4253	
8	6	3916	
9	7	1623	
10	9	644	
11	12	133	

Analysis:

1. Most of the orders are single installment payments that means customers pay one time.
2. Higher installment orders are less frequent, which may indicate higher-value purchases are less.